Aprendizaje Autónomo

Información General	
Facultad:	FEIRNNR
Carrera:	Computación
Semestre y paralelo:	3
Periodo Académico:	2026-2027
Estudiante:	Diyer Arley Torres Troya
Enlace Ejercicios (GitHub/GitLab):	https://github.com/diyertorres-commits/repositorio_base-datos.git
Materia:	Base de datos
Docente Encargado:	EDWIN RENE GUAMAN QUINCHE

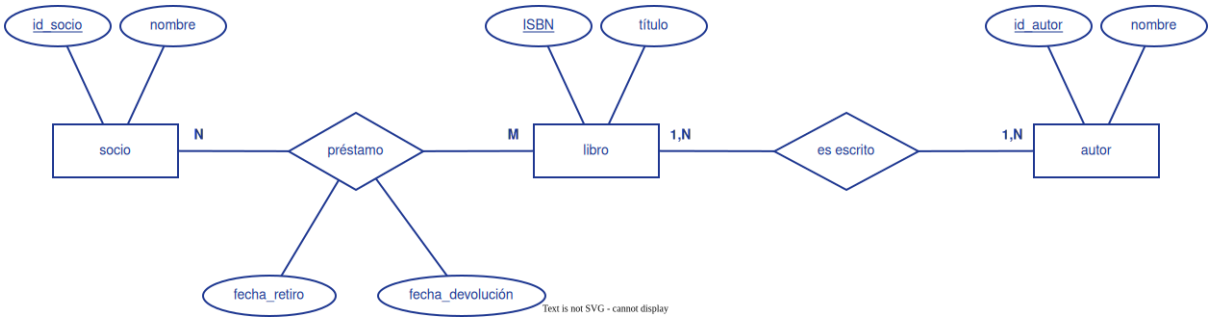
Instrucciones

Agregar la evidencia correspondiente para cada uno de los siguientes ejercicios.

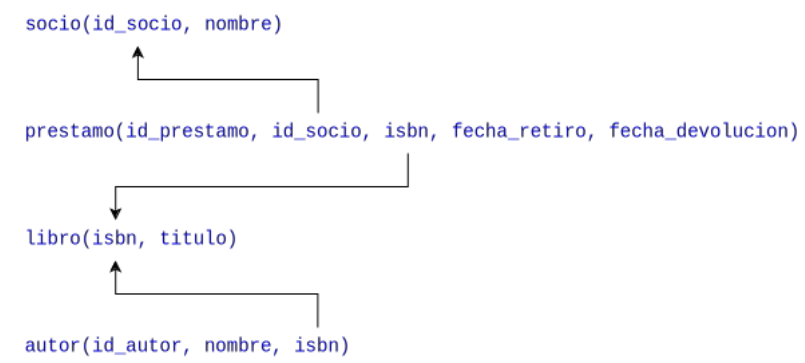
- *Imagen del diagrama Entidad Relación*

Ejemplo 1: Sistema de Biblioteca

Una biblioteca registra libros y autores. Un libro puede tener varios autores y un autor varios libros. Los socios toman libros prestados; de cada préstamo interesan la fecha de retiro y la de devolución.



Ejercicio 1: Sistema de Biblioteca – Modelo relacional




```sql
CREATE TABLE socio (
   id_socio VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

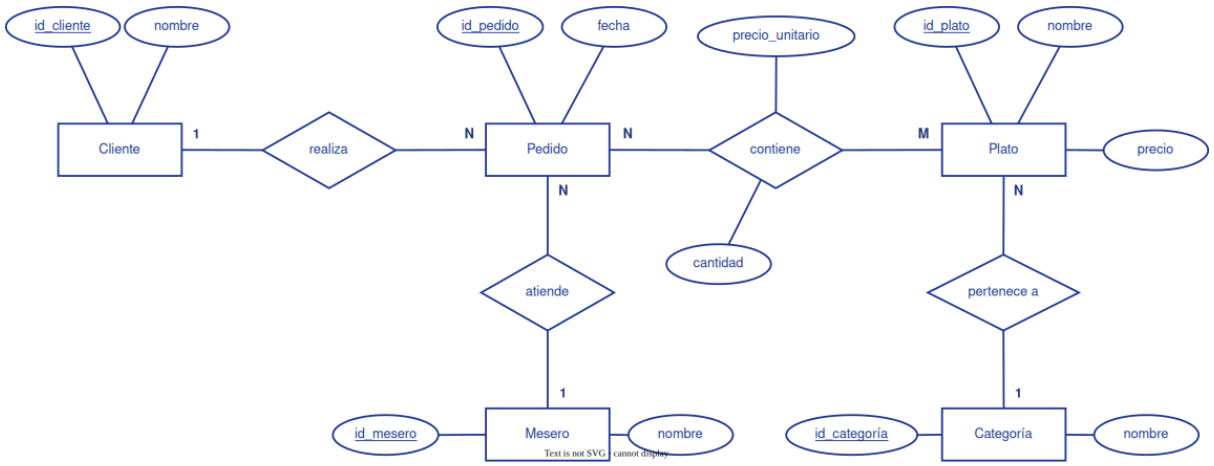
CREATE TABLE libro (
   isbn VARCHAR(20) PRIMARY KEY,
   titulo VARCHAR(150) NOT NULL
);

CREATE TABLE prestamo (
   id_prestamo VARCHAR(15) PRIMARY KEY,
   id_socio VARCHAR(15) NOT NULL,
   isbn VARCHAR(20) NOT NULL,
   fecha_retiro DATE NOT NULL,
   fecha_devolucion DATE,
   FOREIGN KEY (id_socio) REFERENCES socio(id_socio),
   FOREIGN KEY (isbn) REFERENCES libro(isbn)
);

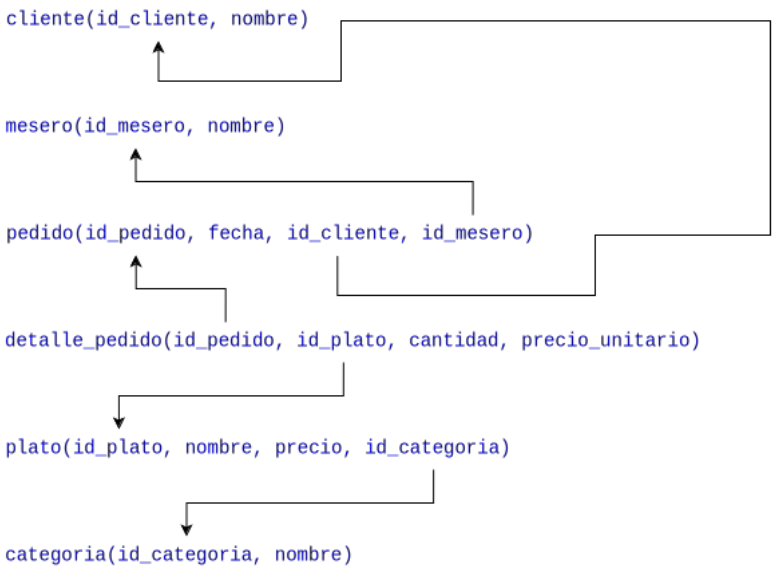
CREATE TABLE autor (
   id_autor VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   isbn VARCHAR(20) NOT NULL,
   FOREIGN KEY (isbn) REFERENCES libro(isbn)
);
```

Ejemplo 2: Pedidos de Restaurante

Los clientes realizan pedidos que contienen varios platos con distinta cantidad. Los platos se agrupan en categorías. Cada pedido lo atiende un único mesero. El precio del plato debe registrarse tal como estaba al momento del pedido, aunque luego cambie.



Ejercicio 2: Pedidos de Restaurante – Modelo relacional




```sql
CREATE TABLE cliente (
   id_cliente VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

CREATE TABLE mesero (
   id_mesero VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
```

```
);

CREATE TABLE categoria (
   id_categoria VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

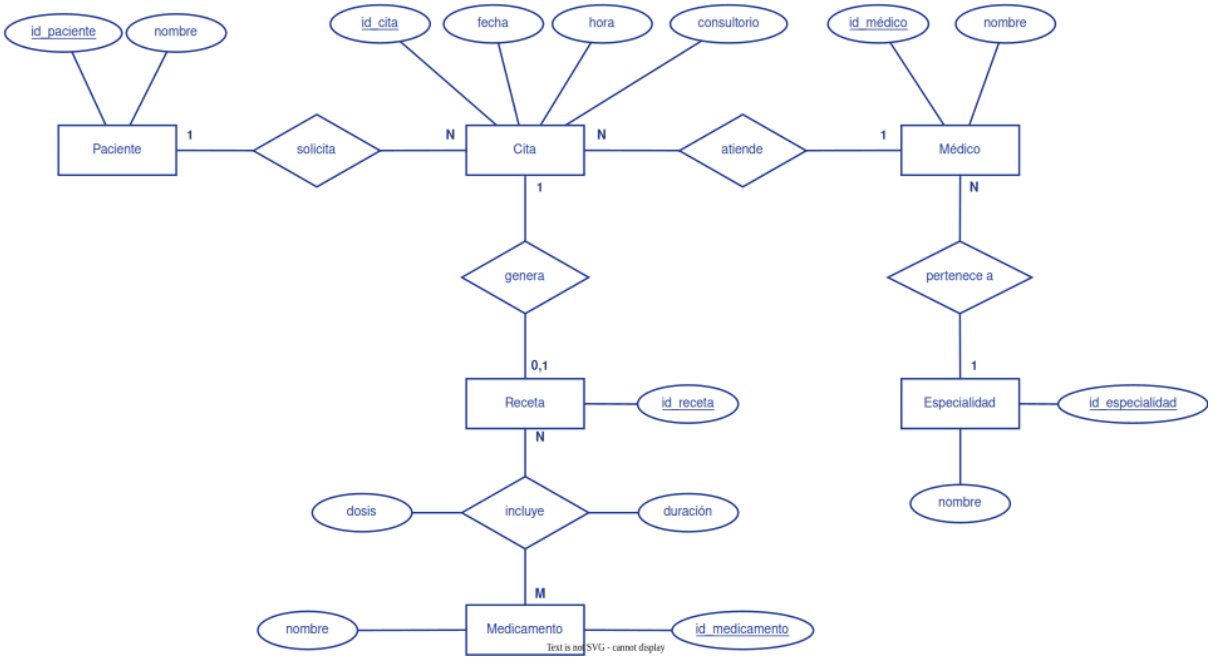
CREATE TABLE plato (
   id_plato VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   precio DECIMAL(10,2) NOT NULL,
   id_categoria VARCHAR(15) NOT NULL,
   FOREIGN KEY (id_categoria) REFERENCES categoria(id_categoria)
);

CREATE TABLE pedido (
   id_pedido VARCHAR(15) PRIMARY KEY,
   fecha DATE NOT NULL,
   id_cliente VARCHAR(15) NOT NULL,
   id_mesero VARCHAR(15) NOT NULL,
   FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente),
   FOREIGN KEY (id_mesero) REFERENCES mesero(id_mesero)
);

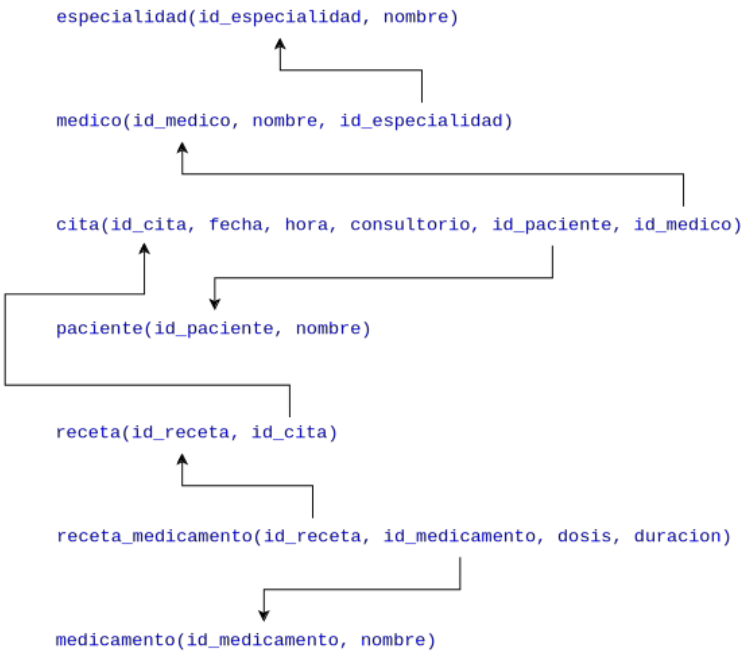
CREATE TABLE detalle_pedido (
   id_pedido VARCHAR(15) NOT NULL,
   id_plato VARCHAR(15) NOT NULL,
   cantidad INT NOT NULL,
   precio_unitario DECIMAL(10,2) NOT NULL,
   PRIMARY KEY (id_pedido, id_plato),
   FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
   FOREIGN KEY (id_plato) REFERENCES plato(id_plato)
);
```

Ejemplo 3: Clínica Médica

Los pacientes solicitan citas; cada cita la atiende un médico a un paciente en fecha, hora y consultorio. De algunas citas se genera una receta que incluye medicamentos con dosis y duración. Cada médico pertenece a una especialidad.



Ejercicio 3: Clínica Médica – Modelo relacional




```sql
CREATE TABLE paciente (
    id_paciente VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE especialidad (
```

```
    id_especialidad VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE medico (
    id_medico VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    id_especialidad VARCHAR(15) NOT NULL,
    FOREIGN KEY (id_especialidad) REFERENCES
especialidad(id_especialidad)
);

CREATE TABLE medicamento (
    id_medicamento VARCHAR(15) PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE cita (
    id_cita VARCHAR(15) PRIMARY KEY,
    fecha DATE NOT NULL,
    hora TIME NOT NULL,
    consultorio VARCHAR(20) NOT NULL,
    id_paciente VARCHAR(15) NOT NULL,
    id_medico VARCHAR(15) NOT NULL,
    FOREIGN KEY (id_paciente) REFERENCES paciente(id_paciente),
    FOREIGN KEY (id_medico) REFERENCES medico(id_medico)
);

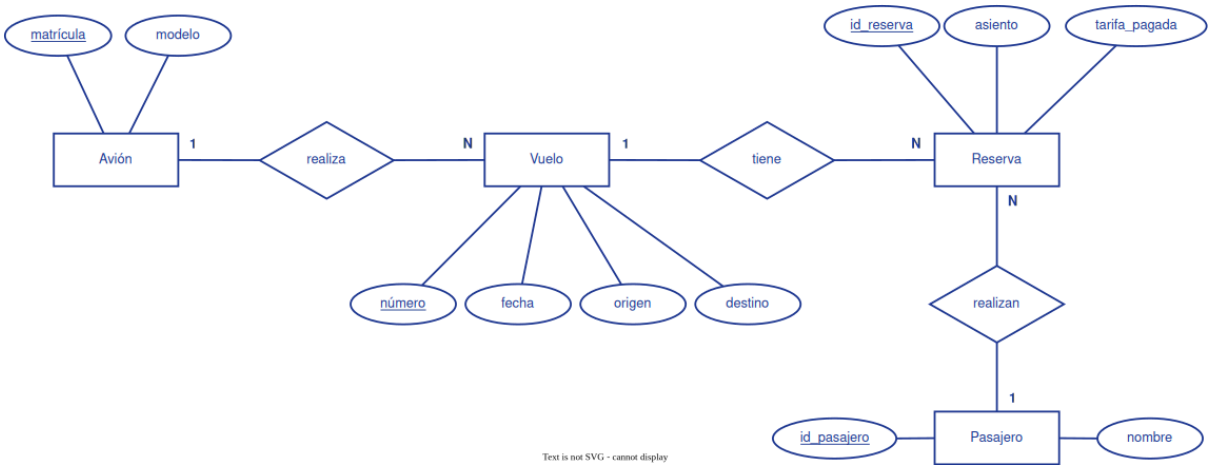
CREATE TABLE receta (
    id_receta VARCHAR(15) PRIMARY KEY,
    id_cita VARCHAR(15) NOT NULL UNIQUE,
    FOREIGN KEY (id_cita) REFERENCES cita(id_cita)
);

CREATE TABLE receta_medicamento (
    id_receta VARCHAR(15) NOT NULL,
    id_medicamento VARCHAR(15) NOT NULL,
    dosis VARCHAR(50) NOT NULL,
    duracion VARCHAR(50) NOT NULL,
    PRIMARY KEY (id_receta, id_medicamento),
    FOREIGN KEY (id_receta) REFERENCES receta(id_receta),
    FOREIGN KEY (id_medicamento) REFERENCES
```

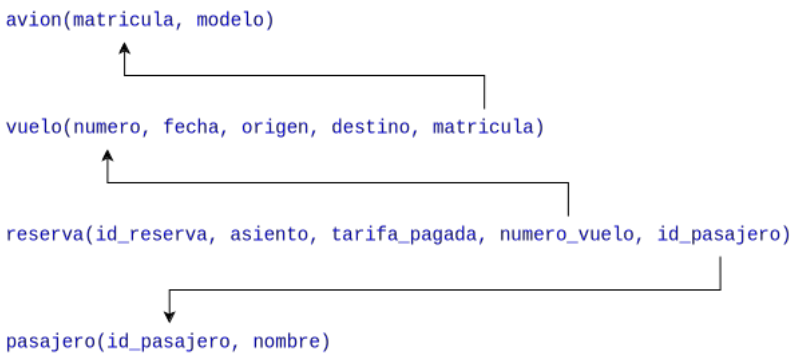
```
medicamento(id_medicamento)
);
```

Ejemplo 4: Sistema de Aerolínea

Una aerolínea opera vuelos (cada uno con número, fecha, origen y destino), cada vuelo lo realiza un avión. Los pasajeros realizan reservas para un vuelo específico, indicando asiento y tarifa pagada en el momento de la reserva.



Ejercicio 4: Sistema de Aerolínea – Modelo relacional




```
CREATE TABLE avion (
   matricula VARCHAR(15) PRIMARY KEY,
   modelo VARCHAR(50) NOT NULL
);

CREATE TABLE pasajero (
   id_pasajero VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
```

```
);

CREATE TABLE vuelo (
   numero VARCHAR(15) PRIMARY KEY,
   fecha DATE NOT NULL,
   origen VARCHAR(60) NOT NULL,
   destino VARCHAR(60) NOT NULL,
   matricula VARCHAR(15) NOT NULL,
   FOREIGN KEY (matricula) REFERENCES avion(matricula)
);

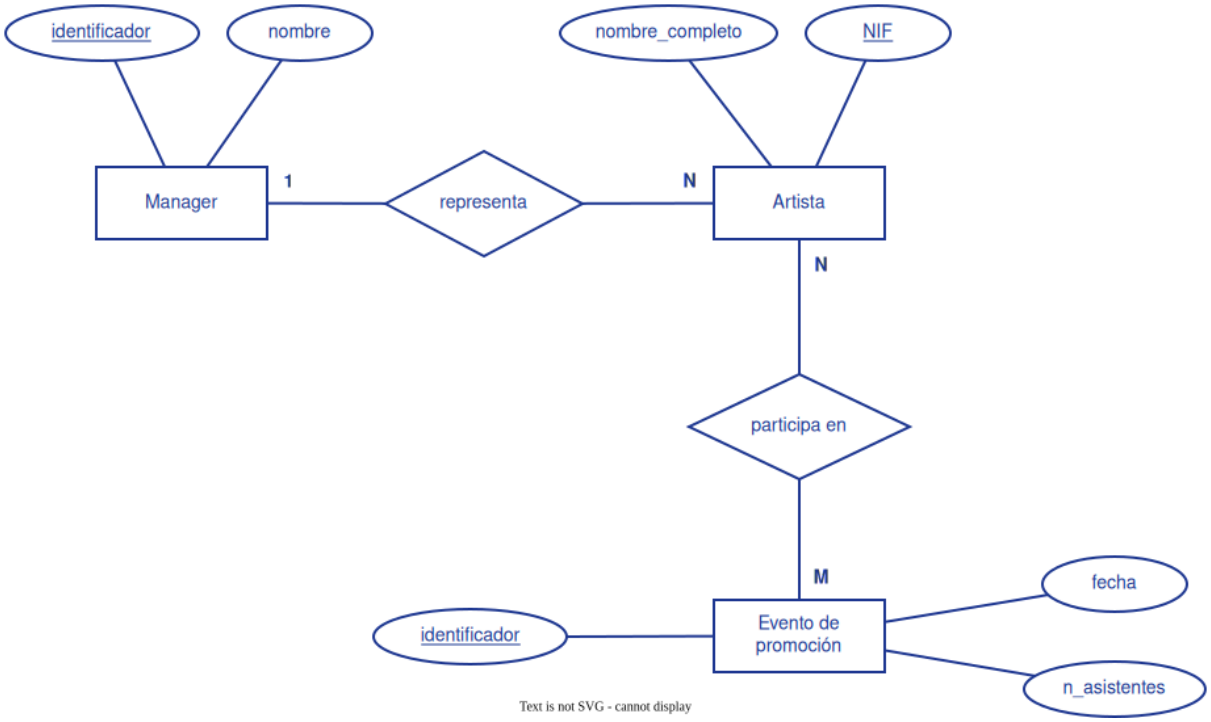
CREATE TABLE reserva (
   id_reserva VARCHAR(15) PRIMARY KEY,
   asiento VARCHAR(5) NOT NULL,
   tarifa_pagada DECIMAL(10,2) NOT NULL,
   numero_vuelo VARCHAR(15) NOT NULL,
   id_pasajero VARCHAR(15) NOT NULL,
   FOREIGN KEY (numero_vuelo) REFERENCES vuelo(numero),
   FOREIGN KEY (id_pasajero) REFERENCES pasajero(id_pasajero)
);
```

Ejercicios Propuestos

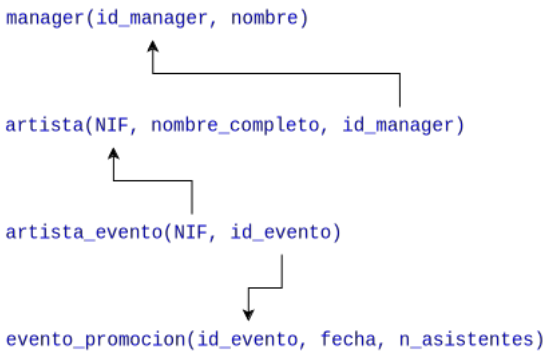
Ejercicio 5: Empresa Discográfica

Una empresa discográfica necesita modelar los datos sobre sus diferentes recursos según las siguientes características:

- Manager: de cada mánager se almacena un identificador de manager y su nombre. Un manager representa a una serie de artistas.
- Artista: un artista es representado por un manager. De los artistas se almacena su nombre completo (usando un único atributo) y su NIF.
- Evento de promoción: los artistas participan en eventos de promoción para dar a conocer sus trabajos. En un evento de promoción pueden participar varios artistas. De un evento se almacena un identificador único, la fecha de celebración y el número de asistentes.



Ejercicio 5: Empresa Discográfica – Modelo relacional




```sql
CREATE TABLE manager (
   id_manager VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

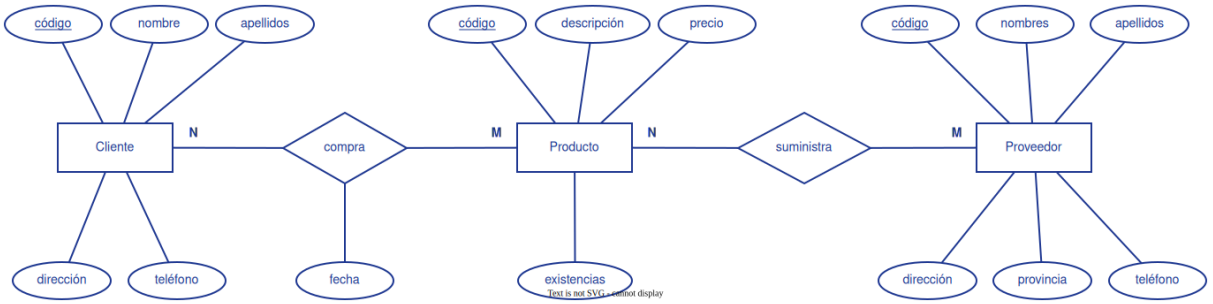
CREATE TABLE artista (
   NIF VARCHAR(15) PRIMARY KEY,
   nombre_completo VARCHAR(150) NOT NULL,
   id_manager VARCHAR(15) NOT NULL,
   FOREIGN KEY (id_manager) REFERENCES manager(id_manager)
);
```

```sql
CREATE TABLE evento_promocion (
   id_evento VARCHAR(15) PRIMARY KEY,
   fecha DATE NOT NULL,
   n_asistentes INT NOT NULL
);

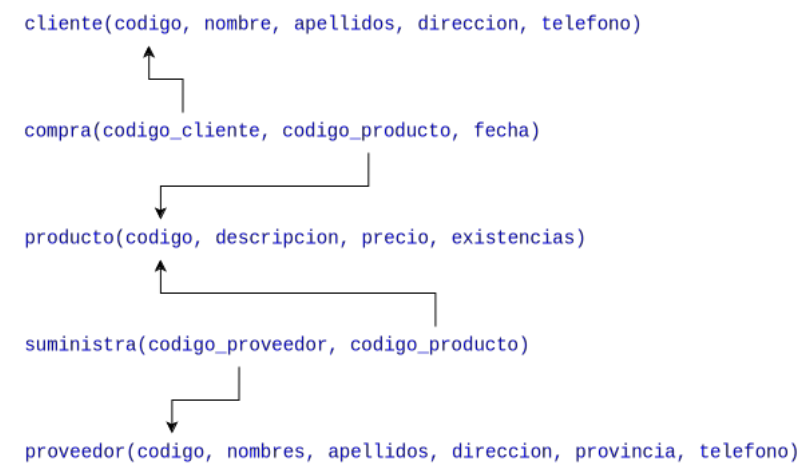
CREATE TABLE artista_evento (
   NIF VARCHAR(15) NOT NULL,
   id_evento VARCHAR(15) NOT NULL,
   PRIMARY KEY (NIF, id_evento),
   FOREIGN KEY (NIF) REFERENCES artista(NIF),
   FOREIGN KEY (id_evento) REFERENCES evento_promocion(id_evento)
);
```

Ejercicio 6: Tienda Informática

Se desea crear una aplicación para la gestión de una tienda informática. La tienda dispone de una serie de productos que se venden a los clientes. De cada producto informático se desea guardar el código, descripción, precio y número de existencias. De cada cliente se desea guardar el código, nombre, apellidos, dirección y número telefónico. Un cliente puede comprar varios productos en la tienda y un mismo producto puede ser comprado por varios clientes. Cada vez que se compre un artículo quedará registrada la compra en la base de datos junto con la fecha en la que se compró el artículo. La tienda tiene contactos con varios proveedores que son los que suministran los productos. Un mismo producto puede ser suministrado por varios proveedores. De cada proveedor se desea guardar el código, nombres, apellidos, dirección, provincia y número de teléfono.



Ejercicio 6: Tienda Informática – Modelo relacional




```sql
CREATE TABLE cliente (
   codigo VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   apellidos VARCHAR(100) NOT NULL,
   direccion VARCHAR(150) NOT NULL,
   telefono VARCHAR(15) NOT NULL
);

CREATE TABLE producto (
   codigo VARCHAR(15) PRIMARY KEY,
   descripcion VARCHAR(200) NOT NULL,
   precio DECIMAL(10,2) NOT NULL,
   existencias INT NOT NULL
);

CREATE TABLE proveedor (
   codigo VARCHAR(15) PRIMARY KEY,
   nombres VARCHAR(100) NOT NULL,
   apellidos VARCHAR(100) NOT NULL,
   direccion VARCHAR(150) NOT NULL,
   provincia VARCHAR(60) NOT NULL,
   telefono VARCHAR(15) NOT NULL
);

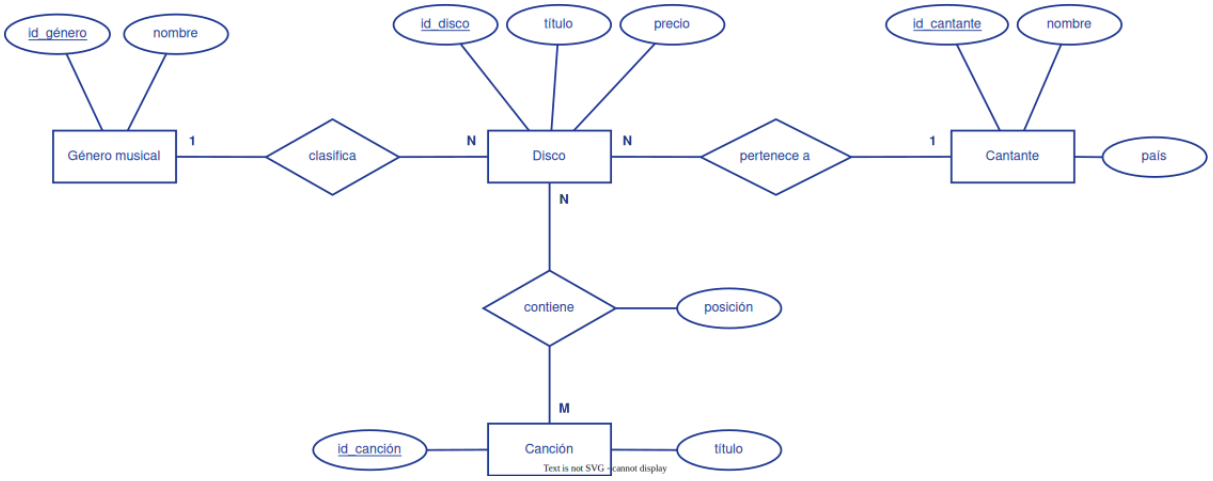
CREATE TABLE compra (
   codigo_cliente VARCHAR(15) NOT NULL,
```

```
    codigo_producto VARCHAR(15) NOT NULL,
    fecha DATE NOT NULL,
    PRIMARY KEY (codigo_cliente, codigo_producto, fecha),
    FOREIGN KEY (codigo_cliente) REFERENCES cliente(codigo),
    FOREIGN KEY (codigo_producto) REFERENCES producto(codigo)
);

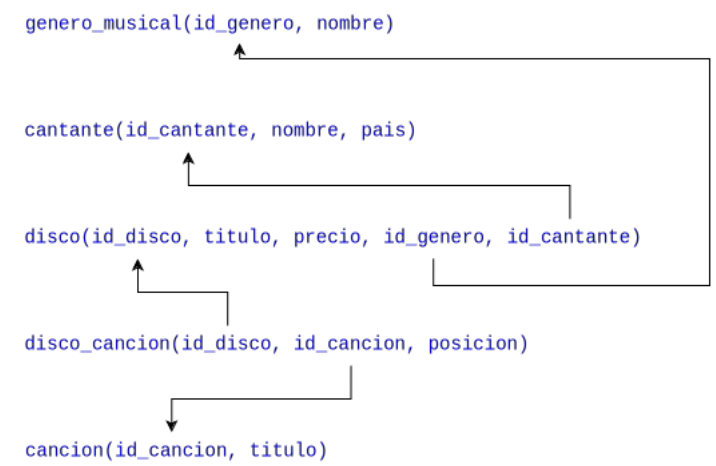
CREATE TABLE suministra (
    codigo_proveedor VARCHAR(15) NOT NULL,
    codigo_producto VARCHAR(15) NOT NULL,
    PRIMARY KEY (codigo_proveedor, codigo_producto),
    FOREIGN KEY (codigo_proveedor) REFERENCES proveedor(codigo),
    FOREIGN KEY (codigo_producto) REFERENCES producto(codigo)
);
```

Ejercicio 7: Discos Musicales

- La tienda vende discos de diferentes géneros musicales y cantantes
- Cada género musical tiene un identificador
- Cada disco tiene titulo, genero musical, precio y cantante
- De un cantante se registra el nombre y su país, Todo disco pertenece a un cantante y un cantante puede tener muchos discos. Además un disco tiene un conjunto de canciones(identificador y título)
- Un disco tiene muchas canciones y una canción puede estar en varios discos. Interesa conocer la posición de una canción en un determinado disco



Ejercicio 7: Discos Musicales – Modelo relacional




```sql
CREATE TABLE genero_musical (
   id_genero VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

CREATE TABLE cantante (
   id_cantante VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   pais VARCHAR(60) NOT NULL
);

CREATE TABLE cancion (
   id_cancion VARCHAR(15) PRIMARY KEY,
   titulo VARCHAR(150) NOT NULL
);

CREATE TABLE disco (
   id_disco VARCHAR(15) PRIMARY KEY,
   titulo VARCHAR(150) NOT NULL,
   precio DECIMAL(10,2) NOT NULL,
   id_genero VARCHAR(15) NOT NULL,
   id_cantante VARCHAR(15) NOT NULL,
   FOREIGN KEY (id_genero) REFERENCES genero_musical(id_genero),
   FOREIGN KEY (id_cantante) REFERENCES cantante(id_cantante)
);

CREATE TABLE disco_cancion (
```

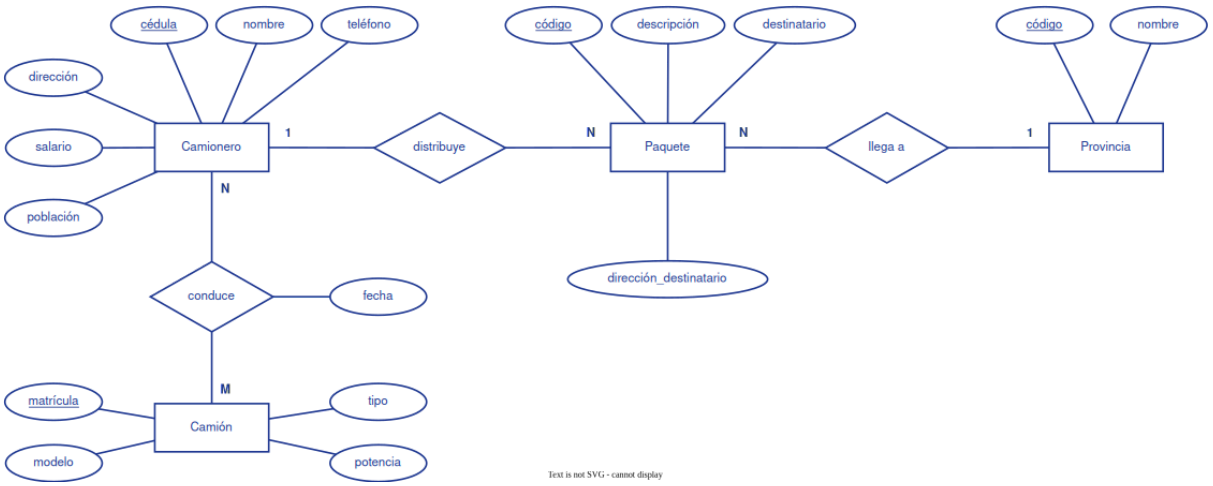
```
    id_disco VARCHAR(15) NOT NULL,
    id_cancion VARCHAR(15) NOT NULL,
    posicion INT NOT NULL,
    PRIMARY KEY (id_disco, id_cancion),
    FOREIGN KEY (id_disco) REFERENCES disco(id_disco),
    FOREIGN KEY (id_cancion) REFERENCES cancion(id_cancion)
);
```

Ejercicio 8: Camiones

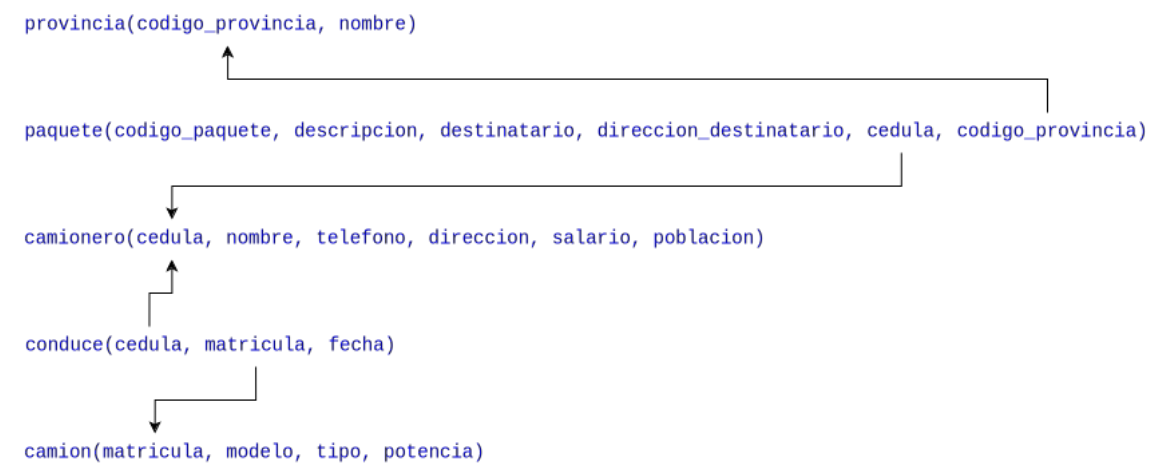
Se desea informatizar la gestión de una empresa de transporte que reparte paquetes por toda España. Los encargados de llevar los paquetes son los camioneros, de los que se quiere guardar la cédula, nombre, teléfono, dirección, salario y población en que vive. De los paquetes transportados interesa conocer el código del paquete, descripción, destinatario y dirección del destinatario.

Un camionero distribuye muchos paquetes, y un paquete sólo puede ser distribuido por un camionero. De las provincias a las que llegan los paquetes interesa guardar el código de provincia y el nombre. Un paquete sólo puede llegar a una provincia. Sin embargo, a una provincia pueden llegar varios paquetes.

De los camiones que llevan los camioneros, interesa conocer la matrícula, modelo, tipo y potencia. Un camionero puede conducir diferentes camiones en fechas diferentes, y un camión puede ser conducido por varios camioneros.



Ejercicio 8: Camiones – Modelo relacional




```sql
CREATE TABLE camionero (
   cedula VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   telefono VARCHAR(15) NOT NULL,
   direccion VARCHAR(150) NOT NULL,
   salario DECIMAL(10,2) NOT NULL,
   poblacion VARCHAR(80) NOT NULL
);

CREATE TABLE provincia (
   codigo_provincia VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL
);

CREATE TABLE camion (
   matricula VARCHAR(15) PRIMARY KEY,
   modelo VARCHAR(50) NOT NULL,
   tipo VARCHAR(50) NOT NULL,
   potencia INT NOT NULL
);

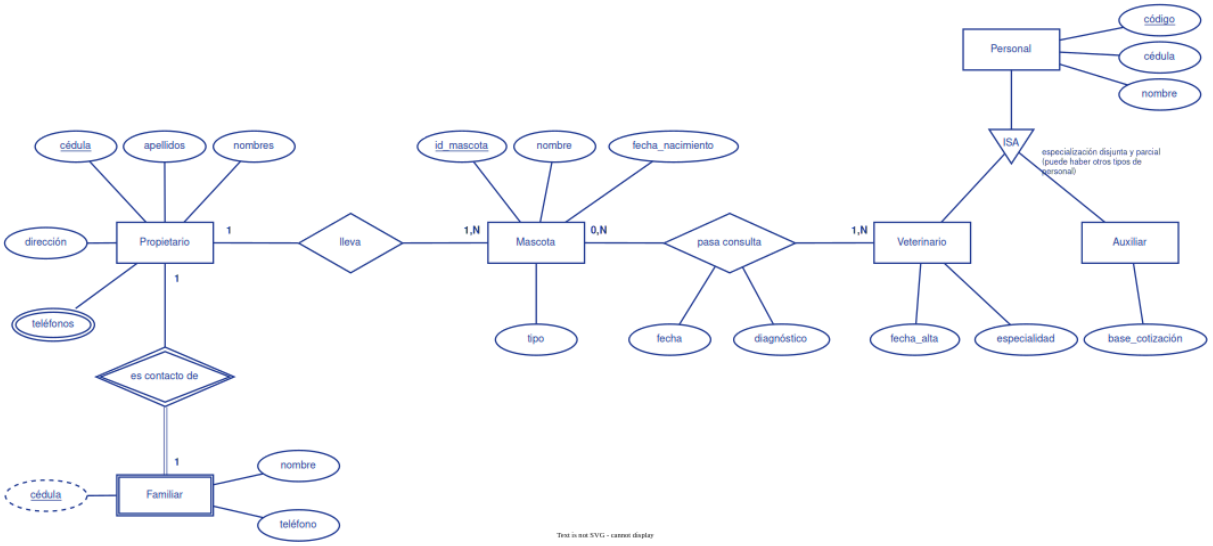
CREATE TABLE paquete (
   codigo_paquete VARCHAR(15) PRIMARY KEY,
   descripcion VARCHAR(200) NOT NULL,
   destinatario VARCHAR(100) NOT NULL,
   direccion_destinatario VARCHAR(150) NOT NULL,
   cedula VARCHAR(15) NOT NULL,
```

```
    codigo_provincia VARCHAR(15) NOT NULL,
    FOREIGN KEY (cedula) REFERENCES camionero(cedula),
    FOREIGN KEY (codigo_provincia) REFERENCES
provincia(codigo_provincia)
);

CREATE TABLE conduce (
    cedula VARCHAR(15) NOT NULL,
    matricula VARCHAR(15) NOT NULL,
    fecha DATE NOT NULL,
    PRIMARY KEY (cedula, matricula, fecha),
    FOREIGN KEY (cedula) REFERENCES camionero(cedula),
    FOREIGN KEY (matricula) REFERENCES camion(matricula)
);
```

Ejercicio 9: Veterinaria

- Del propietario interesa: cédula, apellidos, nombres, dirección , teléfonos
- De la mascota interesa: identificador, nombre, fecha de nacimiento, tipo
- Un propietario puede llevar una o varias mascotas
- Una mascota la lleva uno y solo un propietario
- Del personal de la clínica veterinaria interesa un código, el cédula, nombre
- Entre el personal de la clínica se distingue a los veterinarios y los auxiliares, pudiendo haber otros tipos
    - Del veterinario se guarda además su fecha_alta y especialidad
    - De los auxiliares interesa su base de cotización
- Las mascotas pasan consulta con los veterinarios. De manera que:
    - un veterinario puede pasar consulta a ninguna o a varias mascotas, y
    - una misma mascota de la clínica ha pasado consulta una o más veces en fechas diferentes con varios veterinarios.
    - Al pasar consulta se realiza un diagnóstico.
- Cada propietario tiene un familiar de contacto para casos de emergencia.
    - De este familiar interesa la cédula, nombre y teléfono.
    - Si el propietario se da de alta en la clínica veterinaria, el familiar ya no interesa.
    - El familiar es contacto de un solo propietario



Ejercicio 9: Veterinaria – Modelo relacional

```
familiar(cedula_propietario, cedula_familiar, nombre, telefono)
        ↓
propietario(cedula, apellidos, nombres, direccion)
        ↑
telefono_propietario(cedula, telefono)
        ↑
mascota(id_mascota, nombre, fecha_nacimiento, tipo, cedula_propietario)
        ↑
consulta(id_consulta, id_mascota, codigo_veterinario, fecha, diagnostico)
        ↓
veterinario(codigo, fecha_alta, especialidad)
        ↓
personal(codigo, cedula, nombre)
        ↑
auxiliar(codigo, base_cotizacion)
```

```sql
CREATE TABLE propietario (
    cedula VARCHAR(15) PRIMARY KEY,
    apellidos VARCHAR(100) NOT NULL,
    nombres VARCHAR(100) NOT NULL,
    direccion VARCHAR(150) NOT NULL
);
```

```sql
CREATE TABLE telefono_propietario (
   cedula VARCHAR(15) NOT NULL,
   telefono VARCHAR(15) NOT NULL,
   PRIMARY KEY (cedula, telefono),
   FOREIGN KEY (cedula) REFERENCES propietario(cedula) ON DELETE
CASCADE
);

CREATE TABLE familiar (
   cedula_propietario VARCHAR(15) PRIMARY KEY,
   cedula_familiar VARCHAR(15) NOT NULL,
   nombre VARCHAR(100) NOT NULL,
   telefono VARCHAR(15) NOT NULL,
   FOREIGN KEY (cedula_propietario) REFERENCES propietario(cedula) ON
DELETE CASCADE
);

CREATE TABLE mascota (
   id_mascota VARCHAR(15) PRIMARY KEY,
   nombre VARCHAR(100) NOT NULL,
   fecha_nacimiento DATE NOT NULL,
   tipo VARCHAR(50) NOT NULL,
   cedula_propietario VARCHAR(15) NOT NULL,
   FOREIGN KEY (cedula_propietario) REFERENCES propietario(cedula)
);

CREATE TABLE personal (
   codigo VARCHAR(15) PRIMARY KEY,
   cedula VARCHAR(15) NOT NULL UNIQUE,
   nombre VARCHAR(100) NOT NULL
);

CREATE TABLE veterinario (
   codigo VARCHAR(15) PRIMARY KEY,
   fecha_alta DATE NOT NULL,
   especialidad VARCHAR(100) NOT NULL,
   FOREIGN KEY (codigo) REFERENCES personal(codigo)
);

CREATE TABLE auxiliar (
   codigo VARCHAR(15) PRIMARY KEY,
   base_cotizacion DECIMAL(10,2) NOT NULL,
```

```
    FOREIGN KEY (codigo) REFERENCES personal(codigo)
);

CREATE TABLE consulta (
    id_mascota VARCHAR(15) NOT NULL,
    codigo_veterinario VARCHAR(15) NOT NULL,
    fecha DATE NOT NULL,
    diagnostico VARCHAR(255) NOT NULL,
    PRIMARY KEY (id_mascota, codigo_veterinario, fecha),
    FOREIGN KEY (id_mascota) REFERENCES mascota(id_mascota),
    FOREIGN KEY (codigo_veterinario) REFERENCES veterinario(codigo)
);
```